

Problem Solving Techniques

Muhammad Abdulkariim

Introduction

- **Problem-solving is pattern recognition:** Many problems share similar structures and can be solved using specific techniques.
- **Efficiency matters:** Choosing the right technique can reduce complexity from $O(n^2)$ to $O(n)$ or even $O(\log n)$.
- **Goal of this session:**
 - Learn to **identify problem patterns**.
 - Understand **when to apply specific techniques**.
 - Solve problems **step-by-step** with explanations.

Two Sum | Brute Force

- Given a **sorted array** of integers, find two numbers that add up to a target. Return their indices.

Example:

Input: [2, 7, 11, 15], Target: 9 → Output: [0, 1] (since $2 + 7 = 9$).

- **Brute-Force Approach:**
 - Check every possible pair using **nested loops**.
 - Time Complexity: **$O(n^2)$** (Inefficient for large arrays).
- **Why Brute Force Fails?**
 - It checks all possible pairs, even unnecessary ones (e.g., if the array is sorted, we can do better).

Two-Pointer Technique

- **What it is:**

- Use two pointers (indices) to traverse a data structure (e.g., array or string).

- **When to use it:**

- Problems involving **sorted arrays** or **sequences**.
- Problems where you need to find **pairs**, **subarrays**, or **sequences** that satisfy certain conditions.

- **How to detect the pattern:**

- Look for keywords like "**sorted array**", "**pair of elements**", or "**contiguous sequence**".
- If brute force involves nested loops $O(n^2)$, consider two-pointer for $O(n)$.

Two Sum | Two Pointers

- **Problem:** Given a sorted array, find two numbers that add up to a target.
- **Solution:**
 - Initialize two pointers: $\text{left} = 0$ and $\text{right} = \text{len}(\text{arr}) - 1$.
 - While $\text{left} < \text{right}$:
 - If $\text{arr}[\text{left}] + \text{arr}[\text{right}] == \text{target}$, return $[\text{left}, \text{right}]$.
 - If the sum is too small, move left forward.
 - If the sum is too large, move right backward.
 - If no pair is found, return $[-1, -1]$.
- **Explanation:**
 - The array is sorted, so we can use two pointers to efficiently find the pair without checking all combinations.

Max Sum Subarray of Size K | Brute Force

- **Problem Statement:**

Given an array of integers and a number k , find the **maximum sum** of any contiguous subarray of size k .

Example:

Input: [2, 1, 5, 1, 3, 2], $k=3 \rightarrow$ Output: 9 (from [5, 1, 3]).

- **Brute-Force Approach:**

- Calculate the sum of **every possible subarray** of size k .
- Time Complexity: $O(n \cdot k)$.

- **Why Brute Force Fails?**

- Recalculates sums for overlapping windows (e.g., [2,1,5] and [1,5,1] share [1,5]).

Sliding Window Technique

- **What it is:**

- Maintain a "window" (subarray or substring) and slide it through the data structure.

- **When to use it:**

- Problems involving **contiguous subarrays** or **substrings**.
 - Problems where you need to find **maximum**, **minimum**, or **average** of a window.

- **How to detect the pattern:**

- Look for keywords like "**subarray**", "**substring**", or "**window of size k**".
 - If brute force involves nested loops $O(n^2)$, consider sliding window for $O(n)$.

Max Sum Subarray of Size K | Sliding Window

- **Problem:** Given an array, find the maximum sum of any contiguous subarray of size k .
- **Solution:**
 - Compute the sum of the first window ($\text{arr}[0]$ to $\text{arr}[k-1]$).
 - Slide the window one element at a time:
 - Subtract the element leaving the window.
 - Add the new element entering the window.
 - Track the maximum sum.
- **Explanation:**
 - The sliding window avoids recalculating the sum for overlapping windows, reducing complexity from $O(n \cdot k)$ to $O(n)$.

Longest Substring Without Repeating Characters

- **Problem Statement:**

Given a string, find the **length of the longest substring** without repeating characters.

Example:

Input: "abcabcbb" → Output: 3 (from "abc").

- **Brute-Force Approach:**

- Check **all possible substrings** for uniqueness.
- Time Complexity: $O(n^3)$ (n^2 substrings $\times$ n checks per substring).

- **Why Brute Force Fails?**

- Extremely slow for long strings (e.g., $n=100 \rightarrow \sim 1\text{M}$ checks).

Longest Substring Without Repeating Characters

- **Optimized Solution:** Sliding Window + Hash Set
- **How it works:**
 - Expand the window by moving the right pointer.
 - If a duplicate is found, shrink the window from the left.
 - Track the maximum window size.
 - For input: "abcabcbb"
 - Window: "a" → "ab" → "abc" (max_len=3).
 - Duplicate 'a' → shrink to "bca".
 - Continue until the end.
- **Optimization:**
 - **Time Complexity:** $O(n)$
 - **Space Complexity:** $O(1)$ "Constant Space (26 letters only)"

Detect Cycle in a Linked List | Brute Force

- **Problem:** Determine if a linked list has a cycle.
- **Brute-Force Approach:**
 - Use a **hash set** to track visited nodes.
 - Traverse the list; if a node is revisited, a cycle exists.
 - **Time Complexity:** $O(n)$ time and $O(n)$ space (due to the hash set).
- **Why Brute Force Fails?**
 - Uses **extra memory ($O(n)$)** for the hash set.
 - Can we solve this **without extra space?**

Fast and Slow Pointers

- **What it is:**

- Use two pointers moving at different speeds (e.g., one moves twice as fast).

- **When to use it:**

- Problems involving **linked lists** or **cycle detection**.
- Problems where you need to find the **middle** of a sequence.

- **How to detect the pattern:**

- Look for keywords like "**linked list**", "**cycle**", or "**middle element**".
- If brute force involves extra space or nested loops, consider fast and slow pointers for $O(n)$ time and $O(1)$ space.

Detect Cycle in a Linked List | Fast Slow Pointers

- **Problem:** Determine if a linked list has a cycle.
- **Solution:**
 1. Initialize slow and fast pointers at the head.
 2. Move slow by 1 step and fast by 2 steps.
 3. If fast reaches the end, no cycle exists.
 4. If $\text{slow} == \text{fast}$, a cycle is detected.
 - The fast pointer catches up to the slow pointer if there's a cycle, similar to two runners on a circular track.
- **Optimization:**
 - **Time Complexity:** $O(n)$ (same as brute force).
 - **Space Complexity:** $O(1)$ (no extra memory used).

Subarray Sum Equals K | Brute Force

- **Problem:** Given an array, find the number of subarrays that sum to a target value.
- **Example:**
 - **Input:** nums = [3, 4, -7, 1, 3, 3], k = 7
 - **Output:** 3 (The subarrays are [3, 4], [1, 3, 3] and [3, 4, -7, 1, 3, 3]).
- **Brute-Force Approach:**
 - Check **all possible subarrays** and count those with sum = k.
 - Time Complexity: **$O(n^2)$** (nested loops).
- **Why Brute Force Fails?**
 - For large arrays (e.g., $n = 10^5$), $O(n^2)$ is **too slow** (10^{10} operations).

Prefix Sum Technique

- **What it is:**

- A precomputed array where each element at index i is the sum of all previous elements in the original array.
- Formula: $\text{prefix}[i] = \text{prefix}[i-1] + \text{arr}[i]$

- **When to use it:**

- Problems involving **range sums** or **contiguous subarrays**.
- Optimizes repeated sum queries from $O(n)$ to $O(1)$.

- **How to detect the pattern:**

- Look for keywords like "**subarray sum**", "**range sum**", or "**cumulative**".
- If brute force involves nested loops $O(n^2)$, consider prefix sum for $O(n)$.

Subarray Sum Equals K | Prefix Sum + Hash Map

- **Problem:** Given an array, find the number of subarrays that sum to a target value.
- **Solution:**
 1. Initialize `prefix_sum = 0`, `count = 0`, and `hash_map = {0: 1}`.
 2. Traverse the array:
 1. Update `prefix_sum += nums[i]`.
 2. If `prefix_sum - k` exists in `hash_map`, add its frequency to `count`.
 3. Record `prefix_sum` in `hash_map`.
- **Explanation:**
 - The prefix sum allows us to compute subarray sums in $O(1)$ time, reducing complexity from $O(n^2)$ to $O(n)$.

Step By Step Execution

Index	Num	Prefix Sum	Check prefix_sum - k	Hash Map	Count	Valid Subarrays
-	-	0	-	{0: 1}	0	-
0	3	3	-4 (Not in map)	{0:1, 3:1}	0	-
1	4	7	0 (Frequency=1)	{0:1, 3:1, 7:1}	1	[3,4]
2	-7	0	-7 (Not in map)	{0:2, 3:1, 7:1}	1	-
3	1	1	-6 (Not in map)	{0:2, 1:1, 3:1, 7:1}	1	-
4	3	4	-3 (Not in map)	{0:2, 1:1, 3:1, 4:1, 7:1}	1	-
5	3	7	0 (Frequency=2)	{0:2, 1:1, 3:1, 4:1, 7:2}	3	[1,3,3] And [3,4,- 7,1,3,3]

Key Takeaways

- **Prefix Sum + Hash Map** is the **optimal** approach for subarray sum problems.
- **Steps:**
 - Track cumulative sums.
 - Use a hash map to count valid subarrays in $O(1)$ time.
- **Practice Problems:**
 - LeetCode #560: Subarray Sum Equals K
 - LeetCode #325: Maximum Size Subarray Sum Equals k

Greedy Algorithms

- **What it is:**

- A greedy algorithm makes **locally optimal choices** at each step with the hope of finding a **global optimum**.
- It does not always guarantee the best solution but works for many problems.

- **When to use it:**

- Problems where **local optimization** leads to a **global optimum**.
- Often used in **scheduling, sorting, or selection** problems.

- **How to detect the pattern:**

- Look for keywords like "**maximum**", "**minimum**", or "**optimal**".
- If the problem can be broken into steps where the best local choice leads to the best global solution.

Problem: Activity Selection Problem

- **Problem:** Given a set of activities with start and end times, select the maximum number of non-overlapping activities.
- **Solution:**
 - Sort activities by their **end times**.
 - Select the first activity.
 - For each subsequent activity, if it doesn't overlap with the last selected activity, select it.
- **Explanation:**
 - By always choosing the activity with the **earliest end time**, we maximize the number of non-overlapping activities.
 - This works because selecting the activity that finishes first leaves more room for other activities.

Dynamic Programming (DP)

- **What it is:**

- Break a problem into smaller subproblems and store their solutions to avoid redundant calculations.
- Uses **memoization** (top-down) or **tabulation** (bottom-up) to store intermediate results.

- **When to use it:**

- Problems with **overlapping subproblems** and **optimal substructure**.

- **How to detect the pattern:**

- Look for keywords like "**maximum**", "**minimum**", or "**count possibilities**".
- If the problem can be divided into smaller subproblems that are reused.

Problem: Fibonacci Sequence

- **Problem:** Compute the n -th Fibonacci number.
- **Solution:**
 - Use a DP array to store Fibonacci numbers up to n .
 - Compute each Fibonacci number using the previous two.
- **Explanation:**
 - The DP array avoids redundant calculations, reducing complexity from $O(2^n)$ to $O(n)$.

Binary Search

- **What it is:**
 - Efficiently search for an element or condition in a **sorted array**.
- **When to use it:**
 - Problems involving **sorted data** or **search conditions**.
- **How to detect the pattern:**
 - Look for keywords like "**sorted array**", "**search**", or "**find position**".
 - If brute force involves linear search $O(n)$, consider binary search for $O(\log n)$.

Problem: Search in Rotated Sorted Array

- **Problem:** Given a **rotated sorted array** (e.g., [4,5,6,7,0,1,2]) and a target value target, return the index of target in the array. If not found, return -1.
- **Example:**
 - **Input:** nums = [4,5,6,7,0,1,2], target = 0
 - **Output:** 4 (index of 0).
- **Brute Force Solution:**
 - Use linear search to find the target
 - **Time Complexity:** $O(N)$

Search Rotated Sorted Array | Partial Binary Search

Why Partial Binary Search Works

Property of Rotated Sorted Arrays:

1. The array is **partially sorted** (divided into two sorted subarrays).
2. Example: $[4, 5, 6, 7, 0, 1, 2] \rightarrow [4, 5, 6, 7]$ and $[0, 1, 2]$.

Key Insight:

3. One half of the array is always sorted (even after rotation).
4. Use binary search to determine which half is sorted and whether target lies in that half.

Example Walkthrough

Input: nums = [4,5,6,7,0,1,2], target = 0

Step	left	right	mid	nums[mid]	Action
1	0	6	3	7	7 != 0; left half [4,5,6,7] is sorted. 0 not in [4,7] → search right (left = 4).
2	4	6	5	1	1 != 0; right half [1,2] is sorted. 0 not in [1,2] → search left (right = 4).
3	4	4	4	0	0 == 0 → return 4.

String Techniques

- **What it is:**

- Techniques for solving problems involving **strings** (sequences of characters).

- **Key Techniques:**

- **Pattern Matching.**
 - **Palindrome Detection.**
 - **Anagram Problems.**
 - **String Hashing.**
 - **Trie (Prefix Tree).**

Find a Pattern in a Text | Brute Force

- **Problem Statement:**

Given a text, return all starting indices where the pattern appears in it.

- **Example:**

- **Input:** text = "ABABDABACDABABCABAB", pattern = "ABABC"

- **Brute-force output:** [10] (pattern starts at index 10).

- **Brute-Force Approach:**

- Slide the pattern over the text and check for matches at every position.

- **Time Complexity:** $O(m \cdot n)$, where $m = \text{len}(\text{pattern})$, $n = \text{len}(\text{text})$.

- **Why Brute Force Fails?**

- Inefficient for large texts (e.g., DNA sequences).

- Rechecks already matched characters (e.g., after a mismatch at $i=2$, it restarts from $i=1$).

Pattern Matching (KMP Algorithm)

- **What it is:**

- The **Knuth-Morris-Pratt (KMP)** algorithm is an efficient way to perform pattern matching.
- Preprocesses the pattern to create a **Longest Prefix Suffix (LPS)** array, which helps skip unnecessary comparisons.
- When a mismatch occurs, LPS tells us how much to shift the pattern without rechecking known matches.

- **When to use it:**

- Problems involving **substring search** or **pattern matching**.
- When brute force $O(n \cdot m)$ is too slow, KMP reduces complexity to $O(n+m)$.

- **How to detect the pattern:**

- Look for keywords like "**find substring**", "**pattern matching**", or "**search for occurrence**".

Find a Pattern in a Text | KMP

- **Problem:** Find all occurrences of a pattern in a text.
- **Solution:**
 - Preprocess the pattern to create a **prefix table** (LPS array).
 - The LPS array stores the length of the longest prefix that is also a suffix for each substring of the pattern.
 - Use the prefix table to skip unnecessary comparisons during the search.

Find a Pattern in a Text | KMP (LPS Array)

For pattern = "ABABC":

- LPS = [0, 0, 1, 2, 0]
- Explanation:
 - "A": 0 (no prefix/suffix)
 - "AB": 0
 - "ABA": 1 ("A" is both prefix and suffix)
 - "ABAB": 2 ("AB" is both prefix and suffix)
 - "ABABC": 0

How LPS Optimizes:

- After a mismatch, shift the pattern by $j - \text{LPS}[j-1]$ (where j is the mismatch position in the pattern).

Find a Pattern in a Text | KMP (Execution)

1. Preprocessing:

1. pattern = "ABABC" $\rightarrow$ LPS = [0, 0, 1, 2, 0].

2. Searching:

1. Mismatch at pattern[4] ("C") vs text[14] ("A"):

1. Shift pattern by $j - \text{LPS}[j-1] = 4 - \text{LPS}[3] = 4 - 2 = 2$.

2. Resume comparison at text[14] and pattern[2].

Visualization

Text: A B A B D A B A C D A B A B C A B A B

Pattern: A B A B C (mismatch at C) Shift: ---> A B A B C (shift by 2 using LPS) Match found at index 10!

More Illustration

- Have a quick look on this 5 minutes amazing video:
<https://www.youtube.com/watch?v=ynv7bbcSLKE>

Palindrome Detection

- **What it is:**

- A palindrome is a string that reads the same backward as forward (e.g., "madam" or "racecar").

- **When to use it:**

- Problems involving **palindromic substrings** or **sequences**.
- Common in problems like finding the longest palindromic substring or counting palindromic substrings.

- **How to detect the pattern:**

- Look for keywords like "**palindrome**", "**symmetric string**", or "**read the same backward**".

Problem: Longest Palindromic Substring

- **Problem:** Find the longest palindromic substring in a given string.
- **Solution:**
 - Use a **center expansion** approach.
 - For each character in the string, expand around it to find the longest palindrome.
 - Handle both odd-length and even-length palindromes.

Explanation:

1. Treat each character as the center of the palindrome.
2. Expand outward while the characters on both sides match.

Anagrams

- **What it is:**

- An anagram is a word or phrase formed by rearranging the letters of another (e.g., "listen" and "silent").

- **When to use it:**

- Problems involving **anagram detection** or **grouping**.
- Common in problems like finding all anagrams of a word or grouping anagrams in a list.

- **How to detect the pattern:**

- Look for keywords like **"anagram"**, **"rearrange letters"**, or **"group similar words"**.

Problem: Group Anagrams

- **Problem:** Given a list of words, group the anagrams together.
- **Solution:**
 - Use a hash map to group words with the same sorted representation.
 - The sorted version of an anagram is the same for all words in the group.
- **Explanation:**
 1. For each word, sort its characters to create a key.
 2. Use the key to group words in a hash map.

String Hashing

- **What it is:**

- Use a hash function to map strings to integers for efficient comparison.
- Commonly used in **Rabin-Karp algorithm** for pattern matching.

- **When to use it:**

- Problems involving **string comparison** or **pattern matching**.
- When brute force is too slow, hashing reduces complexity.

- **How to detect the pattern:**

- Look for keywords like "**compare strings**", "**efficient matching**", or "**rolling hash**".

Problem: Rolling Hash

- **Problem:** Find all occurrences of a pattern in a text using hashing.
- **Solution:**
 - Compute the hash of the pattern and the hash of each substring of the text.
 - Compare the hashes to find matches.
- **Explanation:**
 - **Preprocessing:**
 - Compute the hash of the pattern.
 - Compute the hash of the first window of the text.
 - **Rolling Hash:**
 - Slide the window through the text, updating the hash efficiently.
 - If the hash matches, compare the actual strings to avoid false positives.

Trie (Prefix Tree)

- **What it is:**

- A tree-like data structure used to store and retrieve strings efficiently.
- Each node represents a character, and paths represent words.

- **When to use it:**

- Problems involving **prefix matching**, **autocomplete**, or **dictionary storage**.

- **How to detect the pattern:**

- Look for keywords like "**prefix**", "**dictionary**", or "**autocomplete**".

Problem: Implement a Trie

- **Problem:** Implement a trie with insert, search, and startsWith operations.
- **Solution:**
 - Use a TrieNode class to represent each character.
 - Use a Trie class to manage the root node and operations.
- **Explanation:**
 - **Insert:**
 - Traverse the trie, adding nodes for each character in the word.
 - Mark the end of the word with a flag.
 - **Search:**
 - Traverse the trie to check if the word exists.
 - **StartsWith:**
 - Traverse the trie to check if any word starts with the prefix.

Backtracking

- **What it is:**

- A systematic way to explore all possible solutions to a problem by building candidates incrementally and abandoning them as soon as they fail to satisfy constraints.

- **When to use it:**

- Problems involving **permutations, combinations, or exhaustive search**.
- Examples: N-Queens, Sudoku, Subsets, Permutations.

- **How to detect the pattern:**

- Look for keywords like "**all possible solutions**", "**generate permutations**", or "**find all combinations**".

Problem: N-Queens

- **Problem:** Place n queens on an $n \times n$ chessboard such that no two queens threaten each other.
- **Solution:**
 - Use backtracking to place queens row by row.
 - For each row, try placing a queen in every column.
 - If a placement is valid, move to the next row.
 - If no valid placement exists, backtrack and try the next column.
- **Complexity Analysis:**
 - The time complexity of this algorithm is $O(N!)$, where N is the number of queens because there are N possibilities to place the first queen, $N-2$ possibilities for the second queen, $N-4$ possibilities for the third queen, and so on.

```

// Function to check if a queen can be placed at board[row][col]
bool is_under_attack(const std::vector<std::string>& board, int row, int col, int n) {
    for (int i = 0; i < row; ++i) {
        if (board[i][col] == 'Q') return true; // Check column
        if (col - (row - i) >= 0 && board[i][col - (row - i)] == 'Q') return true; // Check diagonal
        if (col + (row - i) < n && board[i][col + (row - i)] == 'Q') return true; // Check anti-diagonal
    }
    return false;
}

// Backtracking function to place queens
void backtrack(std::vector<std::vector<std::string>>& solutions, std::vector<std::string>& board, int row, int n) {
    if (row == n) {
        solutions.push_back(board);
        return;
    }
    for (int col = 0; col < n; ++col) {
        if (!is_under_attack(board, row, col, n)) {
            board[row][col] = 'Q';
            backtrack(solutions, board, row + 1, n);
            board[row][col] = '.';
        }
    }
}

// Main function to solve the N-Queens problem
std::vector<std::vector<std::string>> solve_n_queens(int n) {
    std::vector<std::vector<std::string>> solutions;
    std::vector<std::string> board(n, std::string(n, '.'));
    backtrack(solutions, board, 0, n);
    return solutions;
}

```

Divide and Conquer

- **What it is:**

- Break a problem into smaller subproblems, solve them recursively, and combine their results to solve the original problem.

- **When to use it:**

- Problems that can be divided into **independent subproblems**.
 - Examples: Merge Sort, Quick Sort, Binary Search, Closest Pair of Points.

- **How to detect the pattern:**

- Look for keywords like "**divide into subproblems**", "**recursive solution**", or "**combine results**".

Problem: Merge Sort

- **Problem:** Sort an array using the divide-and-conquer approach.
- **Solution:**
 - Divide the array into two halves.
 - Recursively sort each half.
 - Merge the two sorted halves.